Tech Unicorn

.NET / Golang Backend Engineer Assessment

Intern / Junior Level

Time Allowed	7 days from the date of receipt
Language(s)	C# (.NET) or Golang
Submission	Publicly accessible Git repository (GitHub, GitLab, Bitbucket, etc.)
Required File	README.md with complete setup and run instructions

Problem Statement

InsureZen is a company that digitises and streamlines manual medical insurance claim processing on behalf of multiple insurance companies. Because each medical insurance company uses a different form format, InsureZen is responsible for extracting relevant data from the submitted forms and normalising it into a unified internal representation.

Once a claim has been ingested into the system, it enters a two-stage human review workflow:

- A Maker (InsureZen employee) reviews the extracted claim data, may annotate it with feedback, and submits a recommendation to either approve or reject the claim.
- A Checker (a second InsureZen employee) independently reviews the claim alongside the Maker's recommendation, feedback, and issues the final internal decision.
- Once the Checker has decided, the claim record, including the extracted data, any feedback notes, and the final decision are forwarded to the relevant insurance company for their records and ultimate verdict.

To support this workflow, a set of REST APIs must be built. An existing frontend dashboard consumes these APIs to display analytical summaries and a searchable, paginated history of all claims. You are solely responsible for the backend REST APIs.

Key Notes

Read the following carefully before beginning.

- You are **not** expected to implement OCR or document parsing. Assume that a separate upstream service has already extracted the relevant fields from the raw claim forms and delivers them to your API as standardized, structured input. You are expected to create this standardized input representation.
- InsureZen and its partner insurance companies each process hundreds to thousands of claims per day. Multiple Makers and Checkers will be operating the system concurrently. Your design should account for this.
- You are free to make reasonable assumptions where the problem statement is ambiguous. Document all assumptions in your README. You will be expected to justify them during the review.
- Partial submissions are acceptable. A well-reasoned, clean, and incomplete solution is preferable to a rushed but complete one.

Tasks

Complete the following in order. Each task builds upon the previous.

Task 1 - Requirements Analysis

Read the problem statement carefully and produce a written requirements breakdown. Your output should cover:

- The entities and data points relevant to the insurance claim domain.
- The actors in the system and their respective roles.
- The functional requirements implied by the problem statement.
- Any non-functional requirements you can identify (e.g., concurrency, auditability, data integrity).
- Edge cases and constraints you have identified, along with the assumptions you are making to address them.

Deliverable: Include this analysis in your README or as a separate REQUIREMENTS.md file in your repository.

Task 2 - API Design

Before writing any code, define the API surface for your application. This should include:

- All endpoints with their HTTP methods, paths, request bodies, and response shapes.
- The status codes your API returns under different conditions.
- How your API handles the Maker flow (claim ingestion, review, and recommendation).
- How your API handles the Checker flow (review of a Maker's recommendation and final decision).
- How the forwarding of a completed claim to the insurance company is represented in your model.

Deliverable: Document your API design in your README or as an OpenAPI/Swagger specification.

Task 3 - Implementation

Implement the backend application. Your implementation should satisfy the following:

- The complete Maker flow: retrieval of claims, review, recommendation, and storage of the decision.
- The complete Checker flow: retrieval of claims, review of Maker recommendation, and issuing a final decision.
- A claim history endpoint that is paginated and supports basic filtering (e.g., by status, by insurance company, or by date range).
- Appropriate input validation and meaningful error responses.
- Concurrent-safe handling of claim state transitions (a claim being reviewed by one Maker should not be simultaneously claimable/reviewable by another Maker).

Note: *You are not expected to implement actual communication with an insurance company. A stub or logged record of the forwarding action to your API is sufficient.*

Bonus Tasks.

Tier 1 - Recommended

- Use PostgreSQL as your database.
- Containerise your application with Docker and provide a docker-compose.yml that brings up the full stack (application + database).
- Write at least a small suite of integration or unit tests for the claim state transition logic.

Tier 2 - Advanced

- Create sequence diagrams for the Maker and Checker flows.
- Implement the Maker-facing and Checker-facing services as separate microservices with clearly defined boundaries.
- Add authentication and authorisation using an Identity Provider (IDP) or JWT-based auth, with role-based access control differentiating Makers from Checkers.

Evaluation Criteria

Your submission will be assessed across the following dimensions:

Criterion	What We Are Looking For
Requirements Analysis	Clarity and completeness of identified entities, flows, and edge cases.
API Design	RESTful correctness, appropriate use of HTTP semantics, and consistency.
Code Quality	Readability, structure, naming, and adherence to idiomatic conventions for the chosen language.
Data Modelling	Appropriateness of the schema, use of constraints, and representation of claim state.
Concurrency Handling	Correct and safe handling of concurrent Maker/Checker operations on the same claim.
Error Handling	Informative and consistent error responses; validation of inputs.
Documentation	Quality of the README, setup instructions, and stated assumptions.
Bonus (Optional)	As listed above; assessed only after the core submission is evaluated.

Submission Requirements

- The application must be written in C# (.NET) or Golang.
- The repository must be publicly accessible.
- A README.md must be included with clear instructions on how to build and run the application from scratch.
- The README must also contain your requirements analysis and a record of any assumptions made.
- If using Docker, ensure that docker compose up (or equivalent) is the primary run command.

Note on AI Assistance

The use of AI tools (e.g., GitHub Copilot, ChatGPT, Claude) is permitted as a reference and productivity aid, consistent with how a professional developer would use such tools on the job. However, your submission must reflect your own understanding of the problem.

Submissions where the code, architecture, and documentation appear to have been generated wholly by an AI, with little evidence of independent analysis or decision-making, will be disqualified. During the review interview, you will be asked to walk through and justify the choices made in your submission.